

Notas matematicas complementarias para `nicolas-lm`

Nicolas

Abstract

Estas notas explican matematicamente el codigo del repositorio `nicolas-lm`. El documento no es una introduccion generica a deep learning: cada definicion se conecta con un modulo concreto del codigo, con las dimensiones de sus tensores, con los parametros que aprende y con el experimento de diversidad textual. La exposicion usa como guias a Petersen-Zech para probabilidad, riesgo empirico, optimizacion, redes y Transformers [3]; a Wegner para datasets, regresion, maxima verosimilitud, alta dimension, SVD, perceptron, redes y descenso de gradiente [4]; a Calin para activaciones, costos, entropia cruzada, divergencias, backpropagation, informacion, capacidad y arquitectura [1]; y a Chang et al. para la interpretacion de calidad de datos y tokens efectivos [2].

Contents

1	Mapa entre matematicas y codigo	3
2	Fundamentos de probabilidad	3
2.1	Espacios de probabilidad	3
2.2	Medidas, espacios finitos y ley de una variable	4
2.3	Variables aleatorias y distribuciones	5
2.4	Esperanza, varianza e independencia	5
2.5	Distribuciones condicionales y modelado autoregresivo	6
2.6	Entropia, entropia condicional y maxima verosimilitud	6
2.7	Convergencia e intercambio de limites	6
3	Estadistica y aprendizaje estadistico	7
3.1	Datasets, hipotesis y distribucion de datos	7
3.2	El dataset de <code>nicolas-lm</code>	7
3.3	Train, validation y test	7
3.4	Generalizacion, capacidad y overfitting	8
3.5	Riesgo de Bayes, ERM y descomposicion de generalizacion	8
3.6	Cotas de generalizacion y clases finitas	8
3.7	Numeros de cubrimiento	9
3.8	Compromiso aproximacion-complejidad	9
3.9	Dimension VC y advertencia sobre sobreparametrizacion	10
4	Optimizacion	10
4.1	De ERM a minimizacion	10
4.2	Gradiente y descenso de gradiente	10
4.3	SGD, minibatches y AdamW	11
4.4	Backpropagation	11
5	Deep learning como familia de funciones parametrizadas	11
5.1	Neuronas, capas y activaciones	11
5.2	Tamano, peso compartido y estructura	12
5.3	Activaciones y flujo de gradiente	13
5.4	Softmax, NLL y entropia cruzada	13
5.5	Expresividad no es aprendibilidad	13
6	Tokenizacion y datasets en el repo	14
6.1	CharTokenizer	14
6.2	TokenDataset	14

7	Modelo bigrama	14
7.1	Definicion matematica	14
7.2	Capacidad y limitaciones	14
8	Atencion causal	15
8.1	Tensores de entrada	15
8.2	Queries, keys, values	15
8.3	Mascara causal y softmax	15
9	Transformer pequeño	16
9.1	Embeddings	16
9.2	Bloque pre-norm	16
9.3	Cabeza de lenguaje	16
10	Modelo LLaMA-style	16
10.1	RMSNorm	17
10.2	RoPE	17
10.3	SwiGLU	17
10.4	Atencion LLaMA-style	17
11	Entrenamiento, evaluacion y protocolo experimental	18
11.1	scripts/train.py	18
11.2	scripts/evaluate.py	18
11.3	scripts/run_effective_tokens.py	18
12	Metricas de corpus y tokens efectivos	19
12.1	Entropia empirica	19
12.2	Entropia condicional de bigramas	19
12.3	Distinct-n y gzip	19
12.4	Tokens efectivos	19
13	Interpretacion matematica del resultado	20
14	Auditoria matematica modulo por modulo	20
14.1	tokenizer.py: vocabulario finito y codificacion	20
14.2	data.py: ventanas solapadas y dependencia	20
14.3	corpus_stats.py: estadisticos de informacion	21
14.4	bigram.py: matriz de transicion aprendida	21
14.5	attention.py: atencion como mezcla adaptativa	22
14.6	causal_mask.py: estructura triangular	22
14.7	transformer.py: composicion residual pre-norm	23
14.8	rmsnorm.py: normalizacion por escala	23
14.9	rope.py: posicion como rotacion	23
14.10	swiglu.py: compuerta multiplicativa	24
14.11	llama.py: comparacion estructural con el Transformer pequeño	24
14.12	train.py: problema de optimizacion real	24
14.13	evaluate.py: estimacion de riesgo de test	24
14.14	run_effective_tokens.py: hipotesis experimental	25
14.15	analyze_corpus.py y summarize_results.py	25
15	Derivaciones adicionales	25
15.1	Entropia cruzada y gradiente respecto a logits	25
15.2	Maxima verosimilitud de secuencias autoregresivas	25
15.3	Riesgo, aproximacion, estimacion y optimizacion	26
15.4	Dimension efectiva y alta dimension	26
15.5	Compresion, SVD y embeddings	26
15.6	Backpropagation como regla de la cadena organizada	27

16 Lectura bibliografica aplicada al repo	27
16.1 Petersen–Zech	27
16.2 Wegner	27
16.3 Calin	27
16.4 Chang et al.	28
17 Trazas de dimensiones en una pasada forward	28
17.1 Bigrama	28
17.2 Transformer pequeno	28
17.3 LLaMA-style	29
18 Errores de interpretacion que el documento debe evitar	29
19 Limitaciones matematicas	30
20 Resumen de correspondencias	30

1 Mapa entre matematicas y codigo

El repositorio implementa modelos de lenguaje autoregresivos de nivel caracter. La cadena completa es:

texto crudo $\longrightarrow$ tokens $\longrightarrow$ ventanas $(X_i, Y_i) \longrightarrow p_\theta(Y_i | X_i) \longrightarrow \hat{R}(\theta) \longrightarrow$ metricas de generalizacion y diversidad.

Archivo	Objeto matematico principal
src/nicolasm/tokenizer.py	Bijeccion entre vocabulario finito de caracteres y enteros.
src/nicolasm/data.py	Construccion de ventanas solapadas para prediccion del siguiente token.
src/nicolasm/corpus_stats.py	Estimadores empiricos de entropia, diversidad de n-gramas y compresibilidad.
src/nicolasm/models/bigram.py	Modelo Markoviano de orden uno parametrizado por una matriz de logits.
src/nicolasm/modules/attention.py	Atencion causal escalada con queries, keys, values, mascara y softmax.
src/nicolasm/models/transformer.py	Transformer decoder-only con embeddings, bloques pre-norm y cabeza lineal.
src/nicolasm/models/llama.py	Variante LLaMA-style con RMSNorm, RoPE, SwiGLU y proyecciones sin sesgo.
scripts/train.py	Minimizacion estocastica de entropia cruzada con AdamW.
scripts/evaluate.py	Estimacion descriptiva de perdida de test, perplexidad y brecha.
scripts/run_effective_tokens.py	Protocolo experimental: igualar tamano bruto, entrenar, evaluar y unir metricas.

2 Fundamentos de probabilidad

Petersen–Zech colocan el aprendizaje supervisado sobre una distribucion de datos $\mathcal{D}$. Para que esa frase sea matematica, primero se requiere el lenguaje de espacios medibles y variables aleatorias.

2.1 Espacios de probabilidad

Definicion 2.1 (Sigma-algebra). Sea Ω un conjunto. Una familia $\mathcal{A} \subseteq 2^\Omega$ es una sigma-algebra si:

$$\Omega \in \mathcal{A}, \quad A \in \mathcal{A} \Rightarrow A^c \in \mathcal{A}, \quad A_1, A_2, \dots \in \mathcal{A} \Rightarrow \bigcup_{i=1}^{\infty} A_i \in \mathcal{A}.$$

El par $(\Omega, \mathcal{A})$ se llama espacio medible.

Definicion 2.2 (Probabilidad). Una probabilidad sobre $(\Omega, \mathcal{A})$ es una medida $\mathbb{P} : \mathcal{A} \rightarrow [0, 1]$ tal que $\mathbb{P}(\Omega) = 1$ y, para eventos disjuntos $(A_i)_{i \geq 1}$,

$$\mathbb{P}\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}(A_i).$$

Observacion 2.3 (Interpretacion para lenguaje). *En un modelo idealizado, $\omega \in \Omega$ puede representar una realizacion completa de una fuente textual. El corpus observado en el experimento no es la distribucion misma, sino una secuencia finita producida por algun mecanismo desconocido.*

2.2 Medidas, espacios finitos y ley de una variable

Los apuntes locales `capitulo04.tex` desarrollan primero teoria de la medida antes de hablar de probabilidad. Esta capa es util aqui porque el codigo manipula objetos discretos, pero el lenguaje matematico de aprendizaje usa distribuciones, esperanzas e imagenes directas de medidas.

Definicion 2.4 (Medida positiva). *Sea $(E, \mathcal{E})$ un espacio medible. Una medida positiva es una funcion*

$$\mu : \mathcal{E} \rightarrow [0, \infty]$$

tal que $\mu(\emptyset) = 0$ y, para conjuntos medibles disjuntos $(A_n)_{n \geq 1}$,

$$\mu \left(\bigcup_{n=1}^{\infty} A_n \right) = \sum_{n=1}^{\infty} \mu(A_n).$$

El triplete $(E, \mathcal{E}, \mu)$ se llama espacio de medida.

Observacion 2.5 (El caso del repositorio es discreto). *Para el vocabulario finito $\mathcal{V} = \{0, \dots, m-1\}$, la sigma-algebra natural es $\mathcal{P}(\mathcal{V})$. Para ventanas de longitud B , el espacio de estados es $\mathcal{V}^B$ y la sigma-algebra natural es $\mathcal{P}(\mathcal{V}^B)$. En este caso toda funcion $f : \mathcal{V}^B \rightarrow \mathbb{R}^k$ es medible. Esta observacion evita tecnicismos, pero no elimina la necesidad conceptual de hablar de medidas: entrenar un modelo de lenguaje es estimar distribuciones sobre estos espacios finitos enormes.*

Definicion 2.6 (Imagen directa o pushforward). *Si $\varphi : (E, \mathcal{E}) \rightarrow (F, \mathcal{F})$ es medible y μ es una medida sobre E , la imagen directa $\varphi_{\#}\mu$ se define por*

$$(\varphi_{\#}\mu)(A) = \mu(\varphi^{-1}(A)), \quad A \in \mathcal{F}.$$

Ejemplo 2.7 (La ley de una ventana tokenizada). *Si $W : \Omega \rightarrow \mathcal{V}^B$ envia una realizacion textual ω a la ventana*

$$W(\omega) = (X_t(\omega), \dots, X_{t+B-1}(\omega)),$$

entonces la distribucion de la ventana es el pushforward

$$P_W = W_{\#}\mathbb{P}.$$

El dataset empirico construido por `TokenDataset` reemplaza esa ley desconocida por la medida empirica

$$\hat{P}_N = \frac{1}{N-B} \sum_{i=0}^{N-B-1} \delta_{(x_i, \dots, x_{i+B-1})}.$$

Definicion 2.8 (Medida producto). *Si $(E, \mathcal{E}, \mu)$ y $(F, \mathcal{F}, \nu)$ son espacios de medida sigma-finitos, la medida producto $\mu \otimes \nu$ sobre $(E \times F, \mathcal{E} \otimes \mathcal{F})$ queda caracterizada por*

$$(\mu \otimes \nu)(A \times B) = \mu(A)\nu(B).$$

Observacion 2.9 (Por que importa el producto). *La formulacion iid clasica $S \sim \mathcal{D}^n$ usa exactamente una medida producto:*

$$\mathcal{D}^n = \underbrace{\mathcal{D} \otimes \dots \otimes \mathcal{D}}_{n \text{ veces}}.$$

En el repo, las ventanas provienen de una misma trayectoria textual y no de una extraccion producto independiente. Esta diferencia es la razon formal por la cual las cotas iid de generalizacion deben leerse como marco conceptual, no como certificado exacto para las metricas reportadas.

2.3 Variables aleatorias y distribuciones

Definicion 2.10 (Variable aleatoria). Si $(\Omega, \mathcal{A})$ y $(E, \mathcal{E})$ son espacios medibles, una funcion $X : \Omega \rightarrow E$ es una variable aleatoria si

$$X^{-1}(B) \in \mathcal{A} \quad \text{para todo } B \in \mathcal{E}.$$

Definicion 2.11 (Distribucion). La distribucion de X es la medida P_X sobre E dada por

$$P_X(B) = \mathbb{P}(X \in B).$$

Equivalente y mas estructuralmente,

$$P_X = X_{\#}\mathbb{P}.$$

Ejemplo 2.12 (Tokens). Si el vocabulario del tokenizador tiene tamano m , cada token puede representarse como una variable aleatoria con valores en

$$\mathcal{V} = \{0, \dots, m-1\}.$$

Una secuencia textual se modela entonces como un proceso discreto

$$X_0, X_1, \dots, X_{N-1}, \quad X_t : \Omega \rightarrow \mathcal{V}.$$

2.4 Esperanza, varianza e independendencia

Definicion 2.13. Si $X : \Omega \rightarrow \mathbb{R}^d$ es integrable, su esperanza es

$$\mathbb{E}[X] = \int_{\Omega} X(\omega) d\mathbb{P}(\omega).$$

Si $X : \Omega \rightarrow \mathbb{R}$, su varianza es

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2].$$

Si X, Y son reales, su covarianza es

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])].$$

Observacion 2.14 (Esperanza como integral de Lebesgue). *capitulo04.tex* construye la integral empezando por funciones simples y luego pasando a funciones medibles no negativas por aproximacion monotona. En este proyecto casi todo es finito: para una variable X con valores $\{a_1, \dots, a_r\}$,

$$\mathbb{E}[X] = \sum_{j=1}^r a_j \mathbb{P}(X = a_j).$$

La notacion integral es, sin embargo, la correcta para conectar con riesgo poblacional, leyes de variables aleatorias y resultados de aprendizaje estadistico.

Proposicion 2.15 (Linealidad y Cauchy-Schwarz). Si X, Y son integrables y $a, b \in \mathbb{R}$, entonces

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

Si $X, Y \in L^2(\mathbb{P})$, entonces

$$|\mathbb{E}[XY]| \leq (\mathbb{E}[X^2])^{1/2} (\mathbb{E}[Y^2])^{1/2}.$$

Observacion 2.16. La segunda desigualdad es el caso $p = q = 2$ de Holder, presentado en *capitulo04.tex*. En optimizacion aparece como Cauchy-Schwarz para justificar que $-\nabla F$ es direccion de descenso local; en estadistica controla productos e interacciones entre variables aleatorias.

Definicion 2.17 (Independencia). Dos variables X, Y son independientes si para todo par de conjuntos medibles A, B ,

$$\mathbb{P}(X \in A, Y \in B) = \mathbb{P}(X \in A)\mathbb{P}(Y \in B).$$

Observacion 2.18 (Ventanas no iid). La independendencia es precisamente lo que falla en *TokenDataset*. Si

$$X_i = (x_i, \dots, x_{i+B-1}), \quad X_{i+1} = (x_{i+1}, \dots, x_{i+B}),$$

entonces X_i y X_{i+1} comparten $B-1$ tokens. Por eso el dataset de lenguaje no debe tratarse como una muestra iid ordinaria. Esta advertencia es importante cuando *scripts/evaluate.py* reporta desviacion estandar, error estandar e intervalo descriptivo: esos numeros resumen variabilidad de batches, pero no son intervalos iid formales.

2.5 Distribuciones condicionales y modelado autoregresivo

Definicion 2.19 (Probabilidad condicional discreta). Si $\mathbb{P}(Y = y) > 0$, entonces

$$\mathbb{P}(X = x \mid Y = y) = \frac{\mathbb{P}(X = x, Y = y)}{\mathbb{P}(Y = y)}.$$

Definicion 2.20 (Modelo autoregresivo). Un modelo de lenguaje autoregresivo factoriza la distribucion de una secuencia $(x_0, \dots, x_{T-1})$ como

$$p_\theta(x_0, \dots, x_{T-1}) = \prod_{t=0}^{T-1} p_\theta(x_t \mid x_0, \dots, x_{t-1}).$$

En entrenamiento de next-token prediction se estima, para cada posicion,

$$p_\theta(x_{t+1} \mid x_{\leq t}).$$

2.6 Entropia, entropia condicional y maxima verosimilitud

Definicion 2.21 (Entropia discreta). Si X toma valores en un conjunto finito y $p_i = \mathbb{P}(X = i)$, su entropia es

$$H(X) = - \sum_i p_i \log p_i.$$

Definicion 2.22 (Entropia condicional). Para variables discretas,

$$H(Y \mid X) = - \sum_x \sum_y \mathbb{P}(X = x, Y = y) \log \mathbb{P}(Y = y \mid X = x).$$

Definicion 2.23 (Verosimilitud). Dadas observaciones $z_1, \dots, z_n$ y un modelo parametrico $p_\theta(z)$, la verosimilitud es

$$L(\theta) = \prod_{i=1}^n p_\theta(z_i).$$

Maximizar L equivale a minimizar la negative log-likelihood

$$-\log L(\theta) = \sum_{i=1}^n -\log p_\theta(z_i).$$

Observacion 2.24. Wegner usa la maxima verosimilitud para conectar regresion y supuestos probabilisticos. En modelos de lenguaje, la entropia cruzada implementada por `F.cross_entropy` es exactamente una negative log-likelihood categorica: se penaliza al modelo por asignar poca probabilidad al token observado.

2.7 Convergencia e intercambio de limites

La teoria de la medida de `capitulo04.tex` tambien explica por que los valores esperados se comportan bien bajo limites. En aprendizaje esto aparece cuando se reemplaza una distribucion por medidas empiricas, cuando se promedian perdidas y cuando se justifica pasar de sumas finitas a riesgos esperados.

Proposicion 2.25 (Convergencia dominada, forma util). Sea $Z_n \rightarrow Z$ casi seguramente y supongase que existe $G \in L^1(\mathbb{P})$ tal que $|Z_n| \leq G$ casi seguramente para todo n . Entonces

$$\lim_{n \rightarrow \infty} \mathbb{E}[Z_n] = \mathbb{E}[Z].$$

Observacion 2.26 (Uso conceptual en estas notas). No necesitamos probar convergencia dominada para ejecutar el repo. Pero si queremos explicar rigurosamente por que las perdidas promedio son aproximaciones de riesgos esperados, la teoria de integracion es el lenguaje adecuado. En el caso finito del tokenizer, muchas integrales se reducen a sumas; aun asi, el marco medible evita confundir una suma empirica con la distribucion generadora.

3 Estadística y aprendizaje estadístico

3.1 Datasets, hipótesis y distribución de datos

Definición 3.1 (Dataset etiquetado). Sean X el espacio de entradas y Y el espacio de salidas. Un dataset etiquetado es una colección finita

$$S = \{(x_i, y_i)\}_{i=1}^n \subseteq X \times Y.$$

Definición 3.2 (Hipótesis). Una clase de hipótesis es un conjunto $\mathcal{H}$ de funciones $h : X \rightarrow Y$. En deep learning, $\mathcal{H}$ suele estar parametrizada:

$$\mathcal{H} = \{h_\theta : \theta \in \mathbb{R}^p\}.$$

Definición 3.3 (Riesgo poblacional). Sea $\mathcal{D}$ una distribución sobre $X \times Y$ y sea $\ell : Y \times Y \rightarrow \mathbb{R}_+$ una pérdida. El riesgo de h es

$$R(h) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(h(x), y)].$$

Definición 3.4 (Riesgo empírico). Para un dataset $S = \{(x_i, y_i)\}_{i=1}^n$,

$$\hat{R}_S(h) = \frac{1}{n} \sum_{i=1}^n \ell(h(x_i), y_i).$$

Un minimizador de riesgo empírico satisface

$$h_S \in \operatorname{argmin}_{h \in \mathcal{H}} \hat{R}_S(h).$$

3.2 El dataset de nicolas-lm

El espacio de entrada del repositorio no es $\mathbb{R}^d$ inicialmente. Primero hay texto, luego caracteres, luego ids enteros, y finalmente tensores.

Definición 3.5 (Secuencia tokenizada). Sea $c_0 c_1 \dots c_{N-1}$ un texto. El tokenizador produce

$$x_t = \tau(c_t) \in \{0, \dots, m-1\},$$

donde m es el tamaño del vocabulario.

Definición 3.6 (Ventanas de entrenamiento). Para block size B , `TokenDataset` define

$$X_i = (x_i, x_{i+1}, \dots, x_{i+B-1}) \in \mathcal{V}^B,$$

$$Y_i = (x_{i+1}, x_{i+2}, \dots, x_{i+B}) \in \mathcal{V}^B.$$

El número total de ejemplos de una secuencia de longitud N es $N - B$.

Observación 3.7 (Dimensiones en código). El método `get_batch(batch_size)` devuelve listas que luego `batch_to_tensors` transforma en tensores

$$x, y \in \{0, \dots, m-1\}^{M \times B},$$

donde M es el batch size. En el experimento principal,

$$B = 64, \quad M = 32.$$

3.3 Train, validation y test

Definición 3.8. `train_val_test_split` divide una secuencia en tres bloques contiguos:

$$\text{train} = x_{0:n_1-1}, \quad \text{val} = x_{n_1:n_2-1}, \quad \text{test} = x_{n_2:N-1},$$

donde

$$n_1 = \lfloor \alpha N \rfloor, \quad n_2 = \lfloor (\alpha + \beta) N \rfloor.$$

Por defecto, $\alpha = 0.8$ y $\beta = 0.1$.

Observación 3.9. Train actualiza parámetros. Validation monitorea entrenamiento y hiperparámetros. Test queda reservado para evaluación final. El hecho de que los bloques sean contiguos hace simple el protocolo, pero puede introducir desplazamientos de distribución si el texto cambia de estilo a lo largo del corpus.

3.4 Generalizacion, capacidad y overfitting

Definicion 3.10 (Brecha de generalizacion). *En el reporte del repo,*

$$\text{gap} = \text{test loss} - \text{train loss}.$$

Una brecha positiva indica que el modelo ajusta mejor el conjunto usado para entrenar que el bloque reservado para test.

Observacion 3.11 (Capacidad). *Petersen–Zech formulan la tension entre error de aproximacion y error de generalizacion. En este proyecto, el bigrama tiene baja capacidad; el Transformer y el LLaMA-style tienen mayor capacidad. Mayor capacidad permite representar dependencias mas complejas, pero tambien puede exigir mas datos y mas optimizacion.*

3.5 Riesgo de Bayes, ERM y descomposicion de generalizacion

capitulo03 (1).tex organiza la teoria de generalizacion alrededor de una pregunta precisa: si h_S minimiza el riesgo empirico, cuanto podemos decir del riesgo poblacional $R(h_S)$? Primero se define el punto de referencia ideal.

Definicion 3.12 (Riesgo de Bayes). *El riesgo de Bayes es*

$$R^* = \inf_{h: X \rightarrow Y} R(h),$$

donde el infimo se toma sobre todas las funciones medibles admisibles. Si se restringe a una clase $\mathcal{H}$, aparece el mejor riesgo alcanzable dentro de esa clase:

$$R_{\mathcal{H}}^* = \inf_{h \in \mathcal{H}} R(h).$$

Proposicion 3.13 (Descomposicion ERM). *Sea $h_S \in \mathcal{H}$ un minimizador del riesgo empirico y sea*

$$\varepsilon_{\text{gen}} = \sup_{h \in \mathcal{H}} |R(h) - \hat{R}_S(h)|.$$

Entonces

$$R(h_S) - R^* \leq 2\varepsilon_{\text{gen}} + \left(\inf_{g \in \mathcal{H}} R(g) - R^* \right).$$

Proof. Para cualquier $g \in \mathcal{H}$,

$$R(h_S) - R^* = R(h_S) - \hat{R}_S(h_S) + \hat{R}_S(h_S) - R^*.$$

Como h_S minimiza $\hat{R}_S$, $\hat{R}_S(h_S) \leq \hat{R}_S(g)$. Por tanto,

$$R(h_S) - R^* \leq |R(h_S) - \hat{R}_S(h_S)| + \hat{R}_S(g) - R^*.$$

Sumando y restando $R(g)$,

$$R(h_S) - R^* \leq 2\varepsilon_{\text{gen}} + R(g) - R^*.$$

Tomar infimo sobre $g \in \mathcal{H}$ da la afirmacion. □

Observacion 3.14 (Lectura para este repo). *El primer termino mide generalizacion; el segundo mide aproximacion. En `nicolas-lm`, el bigrama puede tener gran error de aproximacion porque su historia efectiva es solo x_t . El Transformer y LLaMA-style reducen ese sesgo al usar contexto, pero pueden aumentar dificultad de estimacion y optimizacion. Esta es la forma matematica de decir que “mas arquitectura” no garantiza automaticamente menor test loss.*

3.6 Cotas de generalizacion y clases finitas

Definicion 3.15 (Cota de generalizacion). *Una funcion $\kappa(\delta, n)$ es una cota de generalizacion para $\mathcal{H}$ si, con probabilidad al menos $1 - \delta$ sobre $S \sim \mathcal{D}^n$,*

$$\sup_{h \in \mathcal{H}} |R(h) - \hat{R}_S(h)| \leq \kappa(\delta, n).$$

Proposicion 3.16 (Hoeffding mas union bound). *Supongase que $\mathcal{H} = \{h_1, \dots, h_N\}$ es finita y que $\ell(h(x), y) \in [c_1, c_2]$, con $C = c_2 - c_1$. Si $S \sim \mathcal{D}^n$ es iid, entonces con probabilidad al menos $1 - \delta$,*

$$\sup_{h \in \mathcal{H}} |R(h) - \hat{R}_S(h)| \leq C \sqrt{\frac{\log |\mathcal{H}| + \log(2/\delta)}{2n}}.$$

Observacion 3.17 (Por que no se aplica literalmente). *Esta cota usa dos supuestos fuertes: clase finita y muestra iid. Nuestros modelos tienen parametros continuos, y las ventanas del dataset estan solapadas. Aun asi, la formula muestra la intuicion correcta: la generalizacion empeora cuando crece la complejidad efectiva de $\mathcal{H}$, y mejora cuando crece el numero de observaciones efectivamente independientes.*

3.7 Numeros de cubrimiento

Definicion 3.18 (Numero de cubrimiento). *Sea A un subconjunto de un espacio metrico (M, d) . El numero de cubrimiento por radio ε es*

$$\mathcal{G}(A, \varepsilon, (M, d)) = \min \left\{ N : \exists z_1, \dots, z_N \in M \text{ con } A \subseteq \bigcup_{i=1}^N B_\varepsilon(z_i) \right\}.$$

Observacion 3.19 (Interpretacion). *Un cubrimiento reemplaza una clase infinita por una grilla finita de representantes. La teoria de `capitulo03 (1).tex` usa esta idea para pasar de la cota de Hoeffding para clases finitas a clases de redes neuronales con parametros continuos. Si la aplicacion de realizacion*

$$\theta \mapsto h_\theta$$

es Lipschitz en un dominio acotado de parametros, entonces cubrir el espacio de parametros cubre tambien la familia de funciones.

Proposicion 3.20 (Cubrimiento de una caja de parametros). *Para $B, \varepsilon > 0$,*

$$\mathcal{G}([-B, B]^p, \varepsilon, \|\cdot\|_\infty) \leq \left\lceil \frac{B}{\varepsilon} \right\rceil^p.$$

Observacion 3.21 (Conexion con parametros del repo). *El numero p no es solo una cifra administrativa: corresponde al numero de grados de libertad aprendibles. Para el bigrama, $p = m^2$. Para el Transformer y LLaMA-style, p incluye embeddings, proyecciones de atencion, parametros de normalizacion, MLP/SwiGLU y cabeza de lenguaje. La teoria de cubrimientos sugiere que aumentar p aumenta la riqueza de la clase, pero tambien aumenta el termino de complejidad que debe compensarse con mas datos efectivos.*

3.8 Compromiso aproximacion-complejidad

La lectura central de `capitulo03 (1).tex` para nuestro experimento es el compromiso entre capacidad y generalizacion. Una forma esquematica de la cota es

$$R(\Phi_S) - R^* \lesssim \underbrace{\sqrt{\frac{sL \log(mBs)}{n}}}_{\text{complejidad/generalizacion}} + \underbrace{s^{-\alpha}}_{\text{aproximacion}},$$

donde s representa tamano efectivo de red, L profundidad, B cota de pesos, n tamano muestral y α una tasa de aproximacion idealizada.

Observacion 3.22 (Traduccion al experimento high vs medium). *Si el corpus high aumenta diversidad local, tambien puede aumentar la complejidad de la funcion condicional*

$$x_{\leq t} \mapsto p(x_{t+1} \mid x_{\leq t}).$$

Entonces el termino de aproximacion puede mejorar al pasar de bigrama a Transformer, pero el termino de generalizacion/optimizacion puede empeorar si el numero de tokens efectivos no crece lo suficiente. Por eso “high es mas diverso” no implica automaticamente “high produce menor test loss”.

Definicion 3.23 (Subajuste y sobreajuste en estas notas). *Diremos que hay subajuste cuando la perdida de train y test son ambas altas: el modelo no expresa o no optimiza bien la estructura del corpus. Diremos que hay sobreajuste cuando train loss es bajo pero test loss aumenta sustancialmente, es decir, cuando gap domina la interpretacion.*

3.9 Dimension VC y advertencia sobre sobreparametrizaci3n

Definici3n 3.24 (Fragmentaci3n y dimensi3n VC). *Para una clase binaria $\mathcal{H} \subseteq \{h : X \rightarrow \{0, 1\}\}$, un conjunto $\{x_1, \dots, x_k\}$ es fragmentado por $\mathcal{H}$ si para todo vector de etiquetas $(y_1, \dots, y_k) \in \{0, 1\}^k$ existe $h \in \mathcal{H}$ tal que*

$$h(x_i) = y_i, \quad i = 1, \dots, k.$$

La dimensi3n VC es el mayor k para el cual existe un conjunto fragmentado.

Proposici3n 3.25 (Cota VC cl3sica). *Si $\text{VCdim}(\mathcal{H}) = k$, entonces en clasificaci3n binaria con p3rdida 0-1, bajo muestra iid de tama1o $n \geq k$, con probabilidad al menos $1 - \delta$,*

$$|R(h) - \hat{R}_S(h)| \leq \sqrt{\frac{2k \log(en/k)}{n}} + \sqrt{\frac{\log(1/\delta)}{2n}}$$

simultaneamente para todo $h \in \mathcal{H}$.

Observaci3n 3.26 (Por que incluir VC si nuestro objetivo es multiclase). *El repo hace predicci3n categorica con m clases, no clasificaci3n binaria. La dimensi3n VC no es la herramienta exacta para nuestro objetivo, pero si expresa una idea que importa: la capacidad de una clase se mide por los patrones que puede realizar, no solo por el n3mero bruto de par3metros. Para redes profundas, las cotas cl3sicas pueden sugerir pesimismo en regimenes sobreparametrizados; por eso se usan como advertencia conceptual, no como predicci3n num3rica del test loss de este experimento.*

4 Optimizaci3n

4.1 De ERM a minimizaci3n

Una vez elegida la familia p_θ , entrenar significa resolver

$$\hat{\theta} \in \operatorname{argmin}_{\theta \in \mathbb{R}^p} \hat{R}(\theta),$$

donde $\hat{R}$ es la p3rdida promedio sobre batches de ventanas.

Definici3n 4.1 (Funci3n objetivo). *Para un batch $(X, Y) \in \mathcal{V}^{M \times B} \times \mathcal{V}^{M \times B}$, el modelo produce logits*

$$Z_\theta(X) \in \mathbb{R}^{M \times B \times m}.$$

La p3rdida de batch es

$$\hat{R}_B(\theta) = \frac{1}{MB} \sum_{a=1}^M \sum_{t=1}^B -\log \operatorname{softmax}(Z_\theta(X)_{a,t,:})_{Y_{a,t}}.$$

4.2 Gradiente y descenso de gradiente

Definici3n 4.2 (Gradiente). *Si $F : \mathbb{R}^p \rightarrow \mathbb{R}$ es diferenciable,*

$$\nabla F(\theta) = \left(\frac{\partial F}{\partial \theta_1}, \dots, \frac{\partial F}{\partial \theta_p} \right).$$

Para perturbaciones pequenas,

$$F(\theta + v) = F(\theta) + \langle \nabla F(\theta), v \rangle + O(\|v\|^2).$$

Proposici3n 4.3 (Direcci3n local de descenso). *Si $\nabla F(\theta) \neq 0$, la direcci3n unitaria que minimiza la aproximaci3n lineal es*

$$v^* = -\frac{\nabla F(\theta)}{\|\nabla F(\theta)\|}.$$

Proof. Por Cauchy–Schwarz,

$$\langle \nabla F(\theta), v \rangle \geq -\|\nabla F(\theta)\| \|v\|.$$

Si $\|v\| = 1$, la cota se alcanza con $v = -\nabla F(\theta) / \|\nabla F(\theta)\|$. □

Definici3n 4.4 (Descenso de gradiente).

$$\theta_{k+1} = \theta_k - \eta_k \nabla F(\theta_k).$$

El escalar $\eta_k > 0$ es la tasa de aprendizaje.

4.3 SGD, minibatches y AdamW

Definicion 4.5 (Gradiente estocastico). Si

$$F(\theta) = \frac{1}{n} \sum_{i=1}^n F_i(\theta),$$

un minibatch $I_k \subset \{1, \dots, n\}$ define

$$G_k(\theta) = \frac{1}{|I_k|} \sum_{i \in I_k} \nabla F_i(\theta).$$

El update de SGD es

$$\theta_{k+1} = \theta_k - \eta_k G_k(\theta_k).$$

Definicion 4.6 (Adam). Adam mantiene promedios moviles del gradiente y del gradiente cuadrado:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k,$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2)(g_k \odot g_k).$$

La actualizacion idealizada es

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k + \epsilon}}.$$

Observacion 4.7 (AdamW en scripts/train.py). El entrenamiento usa

`torch.optim.AdamW(model.parameters(), lr=3e-4).`

En cada paso el codigo ejecuta:

`loss.backward(), optimizer.step().`

La primera operacion calcula gradientes por backpropagation; la segunda aplica el update adaptativo. La ‘W’ en AdamW indica weight decay desacoplado.

4.4 Backpropagation

Definicion 4.8 (Red como composicion). Sea

$$f_\theta = f_L \circ f_{L-1} \circ \dots \circ f_1.$$

La regla de la cadena da

$$\frac{\partial \ell(f_\theta(x), y)}{\partial \theta_\ell} = \frac{\partial \ell}{\partial f_L} \frac{\partial f_L}{\partial f_{L-1}} \dots \frac{\partial f_{\ell+1}}{\partial f_\ell} \frac{\partial f_\ell}{\partial \theta_\ell},$$

con la notacion adecuada de Jacobianos.

Observacion 4.9. Calin y Petersen–Zech presentan backpropagation como el mecanismo que hace computacionalmente viable entrenar redes profundas. En el repo no se implementa a mano: PyTorch construye el grafo computacional y `loss.backward()` aplica automaticamente la regla de la cadena.

5 Deep learning como familia de funciones parametrizadas

5.1 Neuronas, capas y activaciones

Definicion 5.1 (Neurona). Una neurona es una funcion

$$\nu(x) = \sigma(w^\top x + b),$$

donde $w \in \mathbb{R}^d$, $b \in \mathbb{R}$ y $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ es una activacion.

Definicion 5.2 (Capa). Una capa de ancho k es una aplicacion

$$T : \mathbb{R}^d \rightarrow \mathbb{R}^k, \quad T(x) = \sigma(Wx + b),$$

donde $W \in \mathbb{R}^{k \times d}$, $b \in \mathbb{R}^k$, y σ actua coordenada a coordenada:

$$T(x)_r = \sigma(\langle W_{r,:}, x \rangle + b_r).$$

Esta es la formalizacion de `capitulo02 (1).tex`: una capa apila k neuronas que comparten la misma entrada.

Definicion 5.3 (Red feedforward). Una red feedforward es una composicion

$$f_\theta = T_L \circ \sigma \circ T_{L-1} \circ \cdots \circ \sigma \circ T_1,$$

donde $T_\ell(z) = W_\ell z + b_\ell$.

Definicion 5.4 (Arquitectura, parametros e hiperparametros). Una arquitectura feedforward queda especificada por

$$(\sigma; d_0, d_1, \dots, d_{L+1}),$$

donde d_0 es la dimension de entrada, d_{L+1} la dimension de salida, L el numero de capas ocultas y $d_1, \dots, d_L$ sus anchos. Los parametros aprendibles son las entradas de las matrices $W^{(\ell)}$ y sesgos $b^{(\ell)}$. Los hiperparametros son elecciones fijadas antes de entrenar: profundidad, ancho, activacion, learning rate, block size, batch size y numero de pasos.

Observacion 5.5 (Traduccion al repo). En `nicolas-lm`, `embedding_dim`, `num_heads`, `num_layers`, `block_size` y `dropout` son hiperparametros. Las tablas de `embedding`, `proyecciones de atencion`, `parametros de normalizacion`, `matrices de SwiGLU/MLP` y `lm_head` son parametros. Esta distincion importa: `AdamW` actualiza parametros; no aprende automaticamente el `block size` ni el numero de capas.

Observacion 5.6 (Activaciones). Calin organiza las activaciones en familias sigmoidales, tipo `ReLU` y tipo `bump`. En este repo aparecen `GELU` en el feedforward `Transformer` y `SiLU` dentro de `SwiGLU`. Ambas introducen no linealidad suave, sin la cual una composicion de capas lineales seguiria siendo lineal.

5.2 Tamano, peso compartido y estructura

`capitulo02 (1).tex` introduce una nocion de tamano como numero de parametros aprendibles, permitiendo peso compartido y esparsidad. Si una red tiene pesos y sesgos reunidos en un vector $\theta \in \mathbb{R}^p$, su tamano efectivo es el numero de coordenadas libres de θ , no necesariamente el numero de aristas visibles en un grafo.

Definicion 5.7 (Tamano efectivo). En estas notas escribiremos

$$\text{size}(f_\theta) = \dim(\Theta)$$

para el numero de grados de libertad aprendibles de la familia $\{f_\theta : \theta \in \Theta\}$. Si hay pesos compartidos, varias entradas de matrices corresponden a una sola coordenada de θ . Si hay entradas fijadas en cero, no cuentan como parametros aprendibles.

Observacion 5.8 (Peso compartido en modelos de lenguaje). El modelo `Transformer` no es una red feedforward densa generica aplicada a una ventana aplanada. Sus matrices de atencion y feedforward se aplican a cada posicion temporal con los mismos pesos. Formalmente, si $W_Q \in \mathbb{R}^{d \times d_h}$ define queries, entonces

$$Q_{b,t,:} = H_{b,t,:} W_Q$$

usa el mismo W_Q para todo t . Esto es peso compartido a traves de la dimension temporal. Reduce parametros y, mas importante, impone equivanianza estructural: el mismo patron local puede reconocerse en distintas posiciones.

Observacion 5.9 (Esparsidad estructural por mascara causal). La mascara causal no es esparsidad de parametros, sino esparsidad de conectividad informacional. Obliga

$$\alpha_{t,s} = 0 \quad \text{si } s > t,$$

de modo que la representacion de la posicion t no use informacion futura. La restriccion no reduce directamente $\text{size}(f_\theta)$, pero reduce la clase de dependencias funcionales permitidas.

5.3 Activaciones y flujo de gradiente

Definicion 5.10 (Sigmoide, ReLU, SiLU y GELU). *Las activaciones discutidas en `capitulo02 (1).tex` incluyen*

$$\sigma_{\text{sig}}(x) = \frac{1}{1 + e^{-x}}, \quad \sigma_{\text{ReLU}}(x) = \max\{x, 0\},$$

$$\text{SiLU}(x) = x\sigma_{\text{sig}}(x), \quad \text{GELU}(x) = x\Phi_{\mathcal{N}(0,1)}(x),$$

donde $\Phi_{\mathcal{N}(0,1)}$ es la funcion de distribucion normal estandar.

Observacion 5.11 (Gradientes desvanecientes y explosivos). *Si una composicion profunda satisface, idealizada-mente,*

$$\sup_x |\sigma'(x)| \leq 1 - \delta,$$

entonces los productos de derivadas pueden decrecer como $(1 - \delta)^L$. Si las derivadas o normas Jacobianas son mayores que uno, pueden crecer exponencialmente. Este es el problema de gradiente desvaneciente o explosivo. En el repo, los residuales, LayerNorm/RMSNorm, RoPE y activaciones suaves como GELU o SiLU no eliminan toda dificultad de optimizacion, pero ayudan a mantener una geometria entrenable frente a composiciones profundas.

Observacion 5.12 (Por que SwiGLU usa SiLU). *SwiGLU usa*

$$\text{SiLU}(W_g x) \odot W_u x$$

como compuerta suave. Frente a una ReLU pura, SiLU evita una discontinuidad de la derivada en cero y conserva una senal suave de gradiente. Frente a una sigmoide pura, multiplica por x , lo que permite respuestas no acotadas hacia valores positivos grandes. Esta es una de las razones por las que la variante LLaMA-style del repo no es solo un Transformer con nombres cambiados.

5.4 Softmax, NLL y entropia cruzada

Definicion 5.13 (Softmax). *Para $z \in \mathbb{R}^m$,*

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{r=0}^{m-1} e^{z_r}}.$$

Definicion 5.14 (Negative log-likelihood categorica). *Si la clase correcta es y , la perdida es*

$$\ell(z, y) = -\log \text{softmax}(z)_y.$$

Proposicion 5.15. *Minimizar la entropia cruzada empirica equivale a maximizar la log-verosimilitud del token observado bajo un modelo categorico.*

Proof. Para observaciones y_i con logits z_i ,

$$\prod_i \text{softmax}(z_i)_{y_i}$$

es la verosimilitud. Su logaritmo negativo es

$$-\sum_i \log \text{softmax}(z_i)_{y_i},$$

que coincide con la suma de perdidas de entropia cruzada. □

5.5 Expresividad no es aprendibilidad

Los resultados de aproximacion universal, discutidos por Petersen–Zech, Calin y Wegner, explican que redes suficientemente grandes pueden aproximar clases amplias de funciones. Eso no implica que el entrenamiento de este repo encuentre la funcion ideal. En la practica intervienen:

arquitectura, parametros, datos, optimizador, pasos, ruido de minibatch.

Esta distincion es central para interpretar que el corpus high sea mas diverso pero produzca peor test loss bajo presupuesto fijo.

6 Tokenizacion y datasets en el repo

6.1 CharTokenizer

Definicion 6.1 (Vocabulario finito). Para un texto $c_0 \cdots c_{N-1}$, el vocabulario character-level es

$$C = \{c_t : 0 \leq t < N\}.$$

Si $|C| = m$, `CharTokenizer.from_text` construye una biyeccion

$$\tau : C \rightarrow \{0, \dots, m-1\}.$$

Definicion 6.2 (Codificacion y decodificacion). La codificacion es

$$\text{encode}(c_0 \cdots c_{N-1}) = (\tau(c_0), \dots, \tau(c_{N-1})).$$

La decodificacion aplica τ^{-1} token por token.

Observacion 6.3 (Limitaciones). El tokenizador no aprende morfemas ni subpalabras. Esto hace el experimento limpio matematicamente, porque el vocabulario es exactamente el conjunto de caracteres observados. Pero tambien limita la semantica: el modelo aprende regularidades de caracteres, no unidades lexicas de alto nivel.

6.2 TokenDataset

Definicion 6.4. `TokenDataset(tokens, block_size=B)` implementa

$$i \mapsto (X_i, Y_i)$$

con

$$X_i = (x_i, \dots, x_{i+B-1}), \quad Y_i = (x_{i+1}, \dots, x_{i+B}).$$

Observacion 6.5 (Funcion de perdida por posicion). Aunque Y_i es una secuencia, la perdida se calcula por posicion:

$$\ell_i(\theta) = \frac{1}{B} \sum_{t=0}^{B-1} -\log p_\theta(Y_{i,t} \mid X_{i,\leq t}).$$

La mascara causal garantiza que la posicion t no use $X_{i,t+1}$ ni tokens futuros.

7 Modelo bigrama

7.1 Definicion matematica

Definicion 7.1 (Bigrama). Sea $m = |\mathcal{V}|$. El modelo bigrama aprende una matriz

$$A \in \mathbb{R}^{m \times m}.$$

Para un token actual i , la distribucion del siguiente token es

$$p_\theta(x_{t+1} = j \mid x_t = i) = \text{softmax}(A_{i,:})_j.$$

Observacion 7.2 (Codigo). En `BigramLanguageModel`, `nn.Embedding(vocab_size, vocab_size)` es precisamente una tabla de m filas y m columnas. Consultar el token i devuelve la fila $A_{i,:}$.

7.2 Capacidad y limitaciones

Proposicion 7.3. El bigrama no puede distinguir dos contextos con el mismo ultimo token.

Proof. Por construccion,

$$p_\theta(x_{t+1} \mid x_0, \dots, x_t) = p_\theta(x_{t+1} \mid x_t).$$

Si dos historias terminan en el mismo $x_t = i$, ambas seleccionan la misma fila $A_{i,:}$. □

Observacion 7.4 (Relacion con entropia condicional). *El mejor bigrama posible, en el limite de datos suficientes, se aproxima a la distribucion empirica*

$$\hat{p}(j | i) = \frac{\#\{t : x_t = i, x_{t+1} = j\}}{\#\{t : x_t = i\}}.$$

La dificultad residual de este modelo esta relacionada con

$$H(X_{t+1} | X_t).$$

Si esa entropia es alta, conocer solo el caracter actual deja mucha incertidumbre.

8 Atencion causal

8.1 Tensores de entrada

En CausalSelfAttentionHead, la entrada tiene forma

$$X \in \mathbb{R}^{M \times T \times d},$$

donde M es batch size, $T \leq B$ es longitud de secuencia y d es `embedding_dim`. Cada posicion t tiene un vector $x_t \in \mathbb{R}^d$.

8.2 Queries, keys, values

Definicion 8.1. *Una cabeza de atencion aprende matrices*

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h},$$

y define

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V.$$

Por batch,

$$Q, K, V \in \mathbb{R}^{T \times d_h}.$$

Definicion 8.2 (Scores escalados).

$$S = \frac{QK^\top}{\sqrt{d_h}} \in \mathbb{R}^{T \times T}.$$

La entrada S_{ts} mide compatibilidad entre la query de la posicion t y la key de la posicion s .

Observacion 8.3. *El factor $1/\sqrt{d_h}$ estabiliza la escala de los productos punto. Sin el, si las coordenadas tienen varianza comparable, el producto punto crece con d_h y puede saturar el softmax.*

8.3 Mascara causal y softmax

Definicion 8.4 (Mascara causal). *La mascara $M \in \{0, -\infty\}^{T \times T}$ cumple*

$$M_{ts} = \begin{cases} 0, & s \leq t, \\ -\infty, & s > t. \end{cases}$$

Entonces

$$\alpha_{t,:} = \text{softmax}(S_{t,:} + M_{t,:})$$

asigna masa solo a posiciones pasadas o presentes.

Definicion 8.5 (Salida).

$$O = \alpha V, \quad O_t = \sum_{s \leq t} \alpha_{ts} V_s.$$

Observacion 8.6 (Codigo). `causal_mask(block_size)` crea una matriz triangular inferior booleana. Luego `scores.masked_fill(~mask, -inf)` implementa la mascara M .

9 Transformer pequeño

9.1 Embeddings

Definicion 9.1 (Embedding de tokens). *nn.Embedding(vocab_size, embedding_dim) aprende una matriz*

$$E \in \mathbb{R}^{m \times d}.$$

El token i se representa como $E_i \in \mathbb{R}^d$.

Definicion 9.2 (Embedding posicional absoluto). *El Transformer pequeño aprende*

$$P \in \mathbb{R}^{B \times d}.$$

Para una secuencia $x_0, \dots, x_{T-1}$, la entrada al primer bloque es

$$H_t^{(0)} = E_{x_t} + P_t.$$

9.2 Bloque pre-norm

Definicion 9.3 (LayerNorm idealizada). *Para $x \in \mathbb{R}^d$,*

$$\text{LN}(x) = \gamma \odot \frac{x - \mu(x)\mathbf{1}}{\sqrt{\sigma^2(x) + \epsilon}} + \beta,$$

donde

$$\mu(x) = \frac{1}{d} \sum_{j=1}^d x_j, \quad \sigma^2(x) = \frac{1}{d} \sum_{j=1}^d (x_j - \mu(x))^2.$$

Definicion 9.4 (Bloque Transformer del repo). *El bloque de **TransformerBlock** es*

$$\begin{aligned} \tilde{H}^{(\ell)} &= H^{(\ell)} + \text{MHA}(\text{LN}_1(H^{(\ell)})), \\ H^{(\ell+1)} &= \tilde{H}^{(\ell)} + \text{FFN}(\text{LN}_2(\tilde{H}^{(\ell)})). \end{aligned}$$

Definicion 9.5 (Feedforward GELU). *La red feedforward usada es*

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2,$$

con dimension oculta $4d$.

9.3 Cabeza de lenguaje

Tras L bloques,

$$H^{(L)} \in \mathbb{R}^{M \times T \times d}.$$

La capa final produce logits

$$Z = H^{(L)} W_{\text{lm}} + b_{\text{lm}} \in \mathbb{R}^{M \times T \times m}.$$

El modelo estima

$$p_\theta(x_{t+1} = j \mid x_{\leq t}) = \text{softmax}(Z_{t,:})_j.$$

10 Modelo LLaMA-style

El modelo `LlamaStyleLanguageModel` conserva el objetivo autoregresivo, pero reemplaza tres componentes del Transformer pequeño.

10.1 RMSNorm

Definicion 10.1. Para $x \in \mathbb{R}^d$,

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}.$$

RMSNorm es

$$\text{RMSNorm}(x) = g \odot \frac{x}{\text{RMS}(x)}.$$

Observacion 10.2. A diferencia de *LayerNorm*, *RMSNorm* no resta la media. En el codigo, `variance = x.pow(2).mean(dim=-1, keepdim=True)` y `x * torch.rsqrt(variance + eps)` implementan exactamente esta formula.

10.2 RoPE

Definicion 10.3 (Rotacion por pares). Sea $x \in \mathbb{R}^{d_h}$ con d_h par. Para cada par (x_{2r}, x_{2r+1}) , *RoPE* aplica

$$\begin{pmatrix} x'_{2r} \\ x'_{2r+1} \end{pmatrix} = \begin{pmatrix} \cos \alpha_{t,r} & -\sin \alpha_{t,r} \\ \sin \alpha_{t,r} & \cos \alpha_{t,r} \end{pmatrix} \begin{pmatrix} x_{2r} \\ x_{2r+1} \end{pmatrix},$$

donde

$$\alpha_{t,r} = t \cdot 10000^{-2r/d_h}.$$

Observacion 10.4 (Posicion relativa). El producto punto entre queries y keys rotadas depende de diferencias de posicion a traves de identidades trigonometricas. Por eso *RoPE* codifica informacion posicional sin sumar un embedding absoluto P_t . En el codigo, `apply_rope` se aplica a q y k , no a v .

10.3 SwiGLU

Definicion 10.5.

$$\text{SiLU}(u) = u \sigma(u), \quad \sigma(u) = \frac{1}{1 + e^{-u}}.$$

El bloque *SwiGLU* implementa

$$\text{SwiGLU}(x) = W_d (\text{SiLU}(W_g x) \odot W_u x).$$

Observacion 10.6. La multiplicacion elemento a elemento crea una compuerta aprendible. Una parte del *feedforward* regula cuanta informacion pasa por la otra. Esto reemplaza el *MLP GELU* del *Transformer* pequeno.

10.4 Atencion LLaMA-style

La atencion LLaMA-style usa proyecciones sin sesgo:

$$q = xW_Q, \quad k = xW_K, \quad v = xW_V,$$

aplica *RoPE* a q, k , calcula

$$\text{softmax} \left(\frac{qk^\top}{\sqrt{d_h}} + M \right) v,$$

y proyecta con W_O , tambien sin sesgo. La estructura residual es

$$\begin{aligned} \tilde{H} &= H + \text{Attn}(\text{RMSNorm}(H)), \\ H^+ &= \tilde{H} + \text{SwiGLU}(\text{RMSNorm}(\tilde{H})). \end{aligned}$$

11 Entrenamiento, evaluacion y protocolo experimental

11.1 scripts/train.py

El script de entrenamiento realiza:

1. lee texto UTF-8;
2. construye τ con `CharTokenizer.from_text`;
3. codifica $x_0, \dots, x_{N-1}$;
4. divide train/val/test;
5. construye `TokenDataset` para train y val;
6. construye el modelo seleccionado;
7. minimiza entropia cruzada con AdamW;
8. guarda checkpoint con parametros, tokenizer y perdidas finales.

Matematicamente, cada paso de entrenamiento toma un batch

$$(X, Y) \in \mathcal{V}^{M \times B} \times \mathcal{V}^{M \times B},$$

calcula

$$Z_\theta = f_\theta(X) \in \mathbb{R}^{M \times B \times m},$$

y minimiza

$$\hat{R}_B(\theta) = \frac{1}{MB} \sum_{a,t} -\log \text{softmax}(Z_{\theta,a,t,:})_{Y_{a,t}}.$$

11.2 scripts/evaluate.py

La evaluacion reconstruye el modelo desde el checkpoint y estima perdida en el split de test. Si $\ell_1, \dots, \ell_K$ son perdidas de batches aleatorios, reporta

$$\bar{\ell} = \frac{1}{K} \sum_{k=1}^K \ell_k,$$
$$s = \sqrt{\frac{1}{K-1} \sum_{k=1}^K (\ell_k - \bar{\ell})^2}, \quad \text{se} = \frac{s}{\sqrt{K}}.$$

Tambien reporta

$$\text{PPL} = \exp(\bar{\ell}), \quad \text{gap} = \bar{\ell} - \text{train loss}.$$

Observacion 11.1. *El intervalo $\bar{\ell} \pm 1.96 \text{se}$ en el script es descriptivo. Debido a ventanas solapadas y batches extraidos de una secuencia dependiente, no debe presentarse como intervalo de confianza iid estricto.*

11.3 scripts/run_effective_tokens.py

El protocolo de tokens efectivos hace tres cosas matematicamente importantes:

1. fuerza igual tamano bruto de corpus, N_{chars} , mediante truncamiento o validacion;
2. calcula metricas de diversidad sobre cada corpus;
3. entrena/evalua los mismos modelos bajo hiperparametros comparables.

Asi, la variable experimental principal no es el tamano bruto, sino la estructura estadistica del texto.

12 Métricas de corpus y tokens efectivos

12.1 Entropía empírica

Definición 12.1 (Distribución empírica de n-gramas). Para un texto $c_0, \dots, c_{N-1}$, el conteo del n-grama $g = (a_1, \dots, a_n)$ es

$$C_n(g) = \#\{t : (c_t, \dots, c_{t+n-1}) = g\}.$$

La frecuencia empírica es

$$\hat{p}_n(g) = \frac{C_n(g)}{N - n + 1}.$$

Definición 12.2 (Entropía empírica de n-gramas).

$$\hat{H}_n = - \sum_g \hat{p}_n(g) \log \hat{p}_n(g).$$

Observación 12.3. *character_entropy* calcula $\hat{H}_1$; *ngram_entropy(text, n)* calcula $\hat{H}_n$. Estas entropías son locales: no capturan directamente coherencia semántica global.

12.2 Entropía condicional de bigramas

Definición 12.4.

$$\hat{H}(X_{t+1} | X_t) = \hat{H}(X_t, X_{t+1}) - \hat{H}(X_t).$$

Observación 12.5. Esta métrica se conecta con el bigrama: si la entropía condicional es alta, un modelo que solo conoce X_t enfrenta alta incertidumbre residual.

12.3 Distinct-n y gzip

Definición 12.6 (Distinct-n).

$$\text{distinct}_n = \frac{\#\{\text{n-gramas distintos}\}}{\#\{\text{n-gramas totales}\}}.$$

Definición 12.7 (Gzip ratio). El *repo* define

$$\text{gzip ratio} = \frac{\text{tamano comprimido}}{\text{tamano original}}.$$

Valores mayores indican menor compresibilidad bajo gzip.

Observación 12.8. Gzip mide redundancia explotable por un compresor específico. No mide calidad literaria, factualidad ni utilidad semántica. Por eso no debe confundirse con la noción completa de calidad de datos de Chang et al.

12.4 Tokens efectivos

Chang et al. proponen que el número bruto de tokens no basta para predecir rendimiento; la calidad de datos modifica la cantidad de información útil para el modelo. En su trabajo aparecen señales como diversidad y syntheticity. Este *repo* no estima syntheticity con un modelo docente. Aísla una dimensión:

diversidad textual empírica.

La hipótesis no es que diversidad sea idéntica a tokens efectivos, sino que podría funcionar como proxy parcial si el modelo puede convertir variación en estructura predictiva.

13 Interpretacion matematica del resultado

El corpus high tiene mayor vocabulario, mayor $\hat{H}_n$, mayor distinct-4 y mayor gzip ratio. Por tanto, es mas diverso segun metricas locales y de compresibilidad. Sin embargo, los modelos pequenos producen mayor test loss en high que en medium.

La interpretacion rigurosa es:

mayor diversidad empirica $\nrightarrow$ menor riesgo de test bajo presupuesto fijo.

La diversidad aumenta el conjunto de patrones posibles. Si la capacidad, el numero de pasos o el sesgo arquitectural no bastan, esa riqueza se manifiesta como mayor dificultad predictiva.

Observacion 13.1 (Punto central). *El resultado negativo no contradice la idea de tokens efectivos. La refina: un token es efectivo solo relativo a un sistema completo*

(datos, modelo, optimizador, presupuesto).

14 Auditoria matematica modulo por modulo

Esta seccion escribe, de manera deliberadamente redundante, la correspondencia entre cada modulo del repo y el objeto matematico que implementa. La redundancia es util: el lector puede leer esta parte como una guia de traduccion entre PyTorch y notacion matematica.

14.1 tokenizer.py: vocabulario finito y codificacion

Definicion 14.1 (Vocabulario observado). *Sea $s = (c_0, \dots, c_{N-1})$ el texto de entrenamiento, visto como una secuencia de caracteres. El tokenizer define*

$$\mathcal{V} = \{c : c = c_t \text{ para algun } 0 \leq t < N\}, \quad m = |\mathcal{V}|.$$

Luego ordena $\mathcal{V}$ y construye dos mapas inversos

$$\phi : \mathcal{V} \rightarrow \{0, \dots, m-1\}, \quad \psi : \{0, \dots, m-1\} \rightarrow \mathcal{V},$$

con $\psi(\phi(c)) = c$ para todo $c \in \mathcal{V}$.

Observacion 14.2 (Entrada, salida y limitacion). *La entrada de `encode` es una cadena $s \in \mathcal{V}^N$ y su salida es*

$$x = (\phi(c_0), \dots, \phi(c_{N-1})) \in \{0, \dots, m-1\}^N.$$

La entrada de `decode` es una secuencia de indices y la salida es la cadena obtenida aplicando ψ coordenada a coordenada. Como no hay token unknown, el modelo queda definido sobre el vocabulario observado. Esto es matematicamente limpio, pero restringe la extrapolacion: un caracter no visto no pertenece al espacio muestral tokenizado del experimento.

Ejemplo 14.3 (Por que character-level no es neutro). *Si el texto contiene las palabras “transformer” y “transformacion”, el tokenizer de caracteres comparte subcadenas a nivel de letras, pero no crea una unidad semantica para prefijos, raices o morfemas. Por tanto, el modelo estima una distribucion sobre $\mathcal{V}^N$, no directamente sobre palabras ni conceptos. Esta eleccion reduce el tamano del vocabulario, pero aumenta la longitud de las dependencias necesarias para representar patrones lexicos.*

14.2 data.py: ventanas solapadas y dependencia

El dataset transforma la secuencia tokenizada $x_0, \dots, x_{N-1}$ en pares

$$X_i = (x_i, \dots, x_{i+B-1}) \in \{0, \dots, m-1\}^B, \quad Y_i = (x_{i+1}, \dots, x_{i+B}) \in \{0, \dots, m-1\}^B,$$

para $0 \leq i \leq N - B - 1$. Aqui $B = \text{block_size}$. La salida de `__getitem__` son tensores de forma $(B,)$, y `batch_to_tensors` los apila en

$$X, Y \in \{0, \dots, m-1\}^{M \times B},$$

donde $M = \text{batch_size}$.

Proposicion 14.4 (Las ventanas no son iid). *Para i y $i + 1$, las ventanas comparten $B - 1$ coordenadas:*

$$X_i = (x_i, x_{i+1}, \dots, x_{i+B-1}), \quad X_{i+1} = (x_{i+1}, \dots, x_{i+B}).$$

Por tanto, X_i y X_{i+1} no son independientes salvo en casos degenerados. Mas aun, las etiquetas tambien se solapan:

$$Y_i = (x_{i+1}, \dots, x_{i+B}), \quad Y_{i+1} = (x_{i+2}, \dots, x_{i+B+1}).$$

Observacion 14.5 (Consecuencia para aprendizaje estadistico). *La teoria ERM clasica suele formularse con muestras iid $(Z_i)_{i=1}^n \sim \mathcal{D}^n$. El repo usa una aproximacion empirica distinta: ventanas extraidas de una unica realizacion textual larga. La perdida promedio sigue siendo una cantidad observable y optimizable, pero las cotas iid deben interpretarse como heuristica o reemplazarse por argumentos para procesos dependientes. Esta es una diferencia matematica importante entre el experimento real y la formulacion idealizada de aprendizaje estadistico.*

14.3 corpus_stats.py: estadisticos de informacion

El archivo `corpus_stats.py` no aprende parametros. Define funciones

$$S : \mathcal{V}^N \rightarrow \mathbb{R}^k$$

que resumen propiedades empiricas del texto. Para caracteres,

$$\hat{p}(a) = \frac{1}{N} \sum_{t=0}^{N-1} \mathbf{1}\{x_t = a\}.$$

La entropia empirica es

$$\hat{H}_1 = - \sum_{a \in \mathcal{V}} \hat{p}(a) \log_2 \hat{p}(a).$$

Para n-gramas $u = (u_1, \dots, u_n)$,

$$\hat{p}_n(u) = \frac{1}{N - n + 1} \sum_{t=0}^{N-n} \mathbf{1}\{(x_t, \dots, x_{t+n-1}) = u\},$$

y

$$\hat{H}_n = - \sum_{u \in \mathcal{V}^n} \hat{p}_n(u) \log_2 \hat{p}_n(u).$$

Observacion 14.6 (Que mide y que no mide). $\hat{H}_n$ mide dispersion empirica de patrones locales. No mide veracidad, coherencia discursiva, dificultad semantica ni utilidad para una arquitectura concreta. distinct_n mide soporte relativo observado, mientras que *gzip ratio* mide compresibilidad por un algoritmo especifico. En la terminologia motivada por Chang et al., estas metricas son proxies parciales de calidad/diversidad, no una medicion completa de tokens efectivos.

14.4 bigram.py: matriz de transicion aprendida

El modelo bigrama contiene una tabla de embeddings

$$A \in \mathbb{R}^{m \times m}.$$

Aunque PyTorch la llama `token_embedding_table`, matematicamente cada fila es un vector de logits:

$$A_{i,:} = (A_{i0}, \dots, A_{i,m-1}).$$

Para una entrada $X \in \{0, \dots, m-1\}^{M \times T}$, el modelo produce

$$Z_{b,t,:} = A_{X_{b,t,:}} \in \mathbb{R}^m.$$

Por tanto, los logits tienen dimension

$$Z \in \mathbb{R}^{M \times T \times m}.$$

Definicion 14.7 (Distribucion bigrama). *La probabilidad asignada al siguiente token es*

$$p_{\theta}(Y_{b,t} = j \mid X_{b,t} = i) = \frac{\exp(A_{ij})}{\sum_{\ell=0}^{m-1} \exp(A_{i\ell})}.$$

Proposicion 14.8 (Capacidad del bigrama). *El bigrama puede representar exactamente cualquier cadena de Markov homogénea de orden uno sobre $\mathcal{V}$ cuyas probabilidades de transición sean positivas. No puede representar dependencias que requieran distinguir dos historias con el mismo último carácter.*

Proof. Dada una matriz de transición P con $P_{ij} > 0$, basta tomar $A_{ij} = \log P_{ij}$. Como cada fila suma uno, $\text{softmax}(A_{i,:}) = P_{i,:}$. Si dos historias h y h' terminan en el mismo token i , el modelo usa la misma fila $A_{i,:}$, luego asigna la misma distribución condicional aunque la distribución real dependa de tokens anteriores. $\square$

14.5 attention.py: atención como mezcla adaptativa

Para una cabeza, la entrada es

$$X \in \mathbb{R}^{M \times T \times d}.$$

Los parámetros aprendidos son

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}$$

sin bias. Las proyecciones son

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

con $Q, K, V \in \mathbb{R}^{M \times T \times d_h}$. Para cada batch b , el score entre posiciones t y s es

$$S_{ts}^{(b)} = \frac{\langle Q_{b,t,:}, K_{b,s,:} \rangle}{\sqrt{d_h}}.$$

La máscara causal reemplaza por $-\infty$ los scores con $s > t$:

$$\tilde{S}_{ts}^{(b)} = \begin{cases} S_{ts}^{(b)}, & s \leq t, \\ -\infty, & s > t. \end{cases}$$

Los pesos son

$$\alpha_{ts}^{(b)} = \frac{\exp(\tilde{S}_{ts}^{(b)})}{\sum_{r=0}^t \exp(\tilde{S}_{tr}^{(b)})}, \quad \sum_{s=0}^t \alpha_{ts}^{(b)} = 1,$$

y la salida

$$O_{b,t,:} = \sum_{s=0}^t \alpha_{ts}^{(b)} V_{b,s,:}.$$

Observacion 14.9 (Interpretación). *Cada vector de salida es una combinación convexa de valores pasados y presentes. La combinación depende de la entrada porque los pesos α_{ts} dependen de Q y K . Esta es la diferencia central con un filtro lineal fijo: la matriz de mezcla se recalcula para cada secuencia.*

14.6 causal_mask.py: estructura triangular

La máscara es la matriz booleana

$$C \in \{0, 1\}^{B \times B}, \quad C_{ts} = \mathbf{1}\{s \leq t\}.$$

En el código se registra como buffer, no como parámetro. Esto significa que no aparece en θ ni recibe gradiente. Su función es imponer una restricción estructural sobre la clase de funciones:

$$f_{\theta}(x_0, \dots, x_{T-1})_t \text{ depende solo de } x_0, \dots, x_t.$$

Sin esta restricción, el entrenamiento de next-token prediction podría mirar tokens futuros y la pérdida dejaría de medir predicción autoregresiva.

14.7 transformer.py: composicion residual pre-norm

El Transformer pequeno usa embeddings de token y posicion:

$$E \in \mathbb{R}^{m \times d}, \quad P \in \mathbb{R}^{B \times d}.$$

Para $X \in \{0, \dots, m-1\}^{M \times T}$,

$$H_{b,t,:}^{(0)} = E_{X_{b,t,:}} + P_{t,:}.$$

Cada bloque aplica

$$\begin{aligned} \tilde{H}^{(\ell)} &= H^{(\ell)} + \text{MHA}(\text{LN}(H^{(\ell)})), \\ H^{(\ell+1)} &= \tilde{H}^{(\ell)} + \text{FFN}(\text{LN}(\tilde{H}^{(\ell)})). \end{aligned}$$

La cabeza de lenguaje es una transformacion afin

$$Z_{b,t,:} = W_{\text{lm}} H_{b,t,:}^{(L)} + b_{\text{lm}}, \quad Z \in \mathbb{R}^{M \times T \times m}.$$

Observacion 14.10 (Papel de los residuales). *Los residuales hacen que cada bloque aprenda una correccion de la representacion, no una representacion completamente nueva. Desde el punto de vista de optimizacion, esto mejora el flujo de gradiente porque existe una ruta directa entre capas. En terminos funcionales, el bloque tiene forma $x \mapsto x + g(x)$.*

14.8 rmsnorm.py: normalizacion por escala

Para $x \in \mathbb{R}^d$, el modulo implementa

$$\text{RMSNorm}(x) = g \odot \frac{x}{\sqrt{d^{-1} \sum_{j=1}^d x_j^2 + \epsilon}}, \quad g \in \mathbb{R}^d.$$

La entrada y salida tienen la misma dimension. El unico parametro aprendido es g . A diferencia de LayerNorm, no se resta la media; por tanto RMSNorm controla la escala cuadratica media, no centra el vector.

Observacion 14.11 (Efecto en el modelo LLaMA-style). *RMSNorm aparece antes de atencion y feedforward:*

$$x \mapsto x + \text{Attn}(\text{RMSNorm}(x)), \quad x \mapsto x + \text{SwiGLU}(\text{RMSNorm}(x)).$$

Asi, los submodulos reciben vectores reescalados, mientras la ruta residual preserva la magnitud acumulada de la representacion.

14.9 rope.py: posicion como rotacion

RoPE actua sobre pares de coordenadas. Si

$$u = (u_0, u_1, \dots, u_{d_h-2}, u_{d_h-1}),$$

se agrupan pares (u_{2r}, u_{2r+1}) . Para una posicion t , se define una matriz de rotacion por bloque

$$R_t = \text{diag}(R(t\omega_0), R(t\omega_1), \dots, R(t\omega_{d_h/2-1})),$$

donde

$$R(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}.$$

El codigo aplica

$$q_t \mapsto R_t q_t, \quad k_s \mapsto R_s k_s.$$

Entonces el producto usado por atencion es

$$\langle R_t q_t, R_s k_s \rangle = q_t^\top R_t^\top R_s k_s = q_t^\top R_{s-t} k_s,$$

porque las rotaciones por angulos se componen restando angulos. Esta identidad es la razon matematica de que RoPE codifique informacion relativa $s - t$ dentro del producto query-key.

14.10 swiglu.py: compuerta multiplicativa

Para $x \in \mathbb{R}^d$, el feedforward LLaMA-style no es simplemente $\sigma(Wx)$. Usa dos proyecciones internas:

$$u = W_u x, \quad g = W_g x, \quad u, g \in \mathbb{R}^h.$$

Luego calcula

$$\text{SwiGLU}(x) = W_d (\text{SiLU}(g) \odot u), \quad \text{SiLU}(a) = a\sigma(a).$$

En tensores, si $X \in \mathbb{R}^{M \times T \times d}$, entonces la salida tiene forma $\mathbb{R}^{M \times T \times d}$. Los parametros son

$$W_u, W_g \in \mathbb{R}^{d \times h}, \quad W_d \in \mathbb{R}^{h \times d},$$

mas sesgos si el modulo local los define. El producto coordenada a coordenada crea una compuerta dependiente de la entrada: algunas coordenadas internas son amplificadas o atenuadas segun $\text{SiLU}(W_g x)$.

14.11 llama.py: comparacion estructural con el Transformer pequeno

El modelo LLaMA-style conserva el objetivo autoregresivo y las formas de entrada y salida:

$$X, Y \in \{0, \dots, m-1\}^{M \times T}, \quad Z_\theta(X) \in \mathbb{R}^{M \times T \times m}.$$

Pero cambia la parametrizacion interna:

Transformer pequeno	LLaMA-style	efecto matematico
positional embedding P_t	RoPE en Q, K	posicion absoluta vs relativa en scores
LayerNorm	RMSNorm	centrado+escala vs solo escala
FFN con activacion simple	SwiGLU	no linealidad con compuerta
proyecciones con bias	varias proyecciones sin bias	menor parametrizacion afin

La diferencia no es cosmetica: define otra clase de hipotesis $\mathcal{H}_{\text{llama}}$, con sesgos inductivos distintos para posicion, escala y mezcla no lineal.

14.12 train.py: problema de optimizacion real

El script elige un modelo f_θ , un corpus, un tokenizer y una particion. La funcion que se minimiza en cada minibatch es

$$\ell_M(\theta) = \frac{1}{MT} \sum_{b=1}^M \sum_{t=1}^T -\log \frac{\exp Z_{\theta,b,t,Y_{b,t}}}{\sum_{j=0}^{m-1} \exp Z_{\theta,b,t,j}}.$$

La actualizacion AdamW puede verse como una transformacion

$$\theta_{k+1} = \mathcal{A}_k(\theta_k, \nabla \ell_M(\theta_k)),$$

donde $\mathcal{A}_k$ mantiene momentos de primer y segundo orden y aplica decaimiento de peso desacoplado. El resultado del entrenamiento no es solo el minimo de una funcion abstracta; depende de la semilla, el orden de minibatches, el learning rate, el numero de iteraciones y la arquitectura.

14.13 evaluate.py: estimacion de riesgo de test

La evaluacion repite `eval_iters` minibatches de test y obtiene perdidas $L_1, \dots, L_r$. El script reporta

$$\bar{L} = \frac{1}{r} \sum_{i=1}^r L_i, \quad s^2 = \frac{1}{r-1} \sum_{i=1}^r (L_i - \bar{L})^2, \quad \text{SE} = \frac{s}{\sqrt{r}}.$$

La perplexidad es

$$\text{PPL} = \exp(\bar{L}).$$

Si la perdida esta en nats, $\exp(\bar{L})$ es el numero efectivo de opciones uniformes equivalentes por prediccion. El gap reportado es

$$\text{gap} = \bar{L}_{\text{test}} - \bar{L}_{\text{train}}.$$

Un gap positivo grande sugiere sobreajuste o desplazamiento entre particiones; un gap pequeno con perdidas altas sugiere subajuste o dificultad intrinseca del corpus para el modelo.

14.14 run_effective_tokens.py: hipotesis experimental

El protocolo compara corpus con presupuesto bruto controlado. Si dos corpus tienen el mismo numero de caracteres/tokens brutos, pero difieren en diversidad, el experimento pregunta si

$$N_{\text{efectivo}}(\text{high}) > N_{\text{efectivo}}(\text{medium})$$

se manifiesta como menor perdida de test. La evidencia observada puede fallar por tres razones matematicamente distintas:

1. La diversidad medida no coincide con informacion util para el objetivo autoregresivo.
2. La informacion util existe, pero el modelo pequeno no tiene capacidad o presupuesto para capturarla.
3. La diversidad aumenta tambien la entropia condicional, de modo que el problema predictivo es mas dificil aunque el corpus sea mas rico.

14.15 analyze_corpus.py y summarize_results.py

`analyze_corpus.py` evalua $S(s)$, el vector de estadisticos de corpus. `summarize_results.py` junta ese vector con metricas de entrenamiento:

$$(S(s), \bar{L}_{\text{train}}, \bar{L}_{\text{test}}, \text{PPL}, \text{gap}).$$

Su papel matematico no es entrenar un modelo, sino construir una tabla para comparar relaciones empiricas:

$$\Delta S = S(s_{\text{high}}) - S(s_{\text{medium}}), \quad \Delta L = L_{\text{high}} - L_{\text{medium}}.$$

El punto critico es no interpretar $\Delta S > 0$ como prueba automatica de $\Delta L < 0$. La relacion entre diversidad y generalizacion esta mediada por arquitectura, optimizacion y distribucion.

15 Derivaciones adicionales

15.1 Entropia cruzada y gradiente respecto a logits

Para un unico ejemplo con clase verdadera y y logits $z \in \mathbb{R}^m$, la perdida es

$$\ell(z, y) = -\log \text{softmax}(z)_y = -z_y + \log \sum_{j=0}^{m-1} \exp z_j.$$

Derivando,

$$\frac{\partial \ell}{\partial z_k} = -\mathbf{1}\{k = y\} + \frac{\exp z_k}{\sum_j \exp z_j} = \text{softmax}(z)_k - \mathbf{1}\{k = y\}.$$

En el repo, esta derivada es la senal que PyTorch propaga desde `F.cross_entropy`. Para el bigrama, si $z = A_{i,:}$, solo la fila correspondiente al token actual recibe gradiente directo en ese ejemplo:

$$\frac{\partial \ell}{\partial A_{ik}} = p_{\theta}(k \mid i) - \mathbf{1}\{k = y\}.$$

Esta formula muestra por que el bigrama aprende frecuencias condicionales: empuja hacia arriba el logit del caracter observado y hacia abajo los demas en proporcion a su probabilidad actual.

15.2 Maxima verosimilitud de secuencias autoregresivas

Un modelo autoregresivo factoriza

$$p_{\theta}(x_0, \dots, x_T) = p(x_0) \prod_{t=0}^{T-1} p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

El entrenamiento ignora el termino inicial y maximiza

$$\sum_{t=0}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

La perdida de next-token prediction es exactamente el negativo normalizado de esta log-verosimilitud. En una ventana del repo, la historia disponible queda truncada a longitud maxima B :

$$p_{\theta}(x_{t+1} \mid x_{\leq t}) \approx p_{\theta}(x_{t+1} \mid x_{\max(0, t-B+1), \dots, x_t}).$$

Por eso `block_size` es un hiperparametro estadistico, no solo una decision computacional: limita la memoria del estimador.

15.3 Riesgo, aproximacion, estimacion y optimizacion

Para una arquitectura fija $\mathcal{H} = \{f_{\theta} : \theta \in \Theta\}$, el exceso de riesgo puede descomponerse informalmente como

$$R(\hat{\theta}) - R^* = \underbrace{R(\theta_{\mathcal{H}}^*) - R^*}_{\text{error de aproximacion}} + \underbrace{R(\hat{\theta}_{\text{ERM}}) - R(\theta_{\mathcal{H}}^*)}_{\text{error de estimacion}} + \underbrace{R(\hat{\theta}) - R(\hat{\theta}_{\text{ERM}})}_{\text{error de optimizacion}}.$$

Esta descomposicion explica por que un corpus high puede dar peor perdida:

1. Si el modelo es demasiado pequeno, aumenta el error de aproximacion.
2. Si la muestra efectiva es baja por dependencia o redundancia, aumenta el error de estimacion.
3. Si el paisaje no convexo es mas dificil o el presupuesto de entrenamiento es insuficiente, aumenta el error de optimizacion.

15.4 Dimension efectiva y alta dimension

Wegner enfatiza que muchos problemas de datos viven en espacios de alta dimension pero poseen estructura exploitable. En este repo, cada ventana

$$X_i \in \{0, \dots, m-1\}^B$$

tiene m^B configuraciones posibles. Incluso para $m = 80$ y $B = 128$, ese espacio es astronomico. El aprendizaje solo es posible porque el texto no ocupa uniformemente $\mathcal{V}^B$; tiene regularidades. Los modelos comparados imponen distintas formas de compresion:

bigrama: $\mathcal{V}^B \rightarrow \mathcal{V}$ usando solo x_t ,

Transformer: $\mathcal{V}^B \rightarrow \mathbb{R}^d$ mediante atencion y composicion,

LLaMA-style: $\mathcal{V}^B \rightarrow \mathbb{R}^d$ con normalizacion, rotacion y gating.

Las metricas de corpus intentan medir parte de esa estructura, pero no la dimension intrinseca completa del proceso textual.

15.5 Compresion, SVD y embeddings

Aunque el repo no calcula SVD durante entrenamiento, la idea de compresion lineal ayuda a interpretar embeddings. La matriz de embeddings

$$E \in \mathbb{R}^{m \times d}$$

asigna a cada token un vector en dimension d . Si se mirara una matriz de coocurrencias $C \in \mathbb{R}^{m \times m}$, una aproximacion de bajo rango

$$C \approx U_r \Sigma_r V_r^{\top}$$

representaria tokens por coordenadas latentes. El embedding aprendido cumple un papel analogo, pero optimizado indirectamente por la perdida autoregresiva y no por reconstruccion cuadratica. Esta conexion justifica incluir herramientas de algebra lineal y compresion al explicar un modelo de lenguaje pequeno.

15.6 Backpropagation como regla de la cadena organizada

Sea el modelo una composicion

$$f_\theta = f_L \circ f_{L-1} \circ \cdots \circ f_1.$$

Si $h_0 = X$, $h_\ell = f_\ell(h_{\ell-1}; \theta_\ell)$ y $\ell = \mathcal{L}(h_L, Y)$, entonces

$$\frac{\partial \ell}{\partial \theta_\ell} = \frac{\partial \ell}{\partial h_\ell} \frac{\partial h_\ell}{\partial \theta_\ell}, \quad \frac{\partial \ell}{\partial h_{\ell-1}} = \frac{\partial \ell}{\partial h_\ell} \frac{\partial h_\ell}{\partial h_{\ell-1}}.$$

PyTorch construye este grafo automaticamente, pero matematicamente el proceso es la regla de la cadena aplicada a embeddings, atencion, normalizaciones, MLPs y cabeza lineal. Calin desarrolla este punto para redes profundas: la dificultad no es solo escribir derivadas locales, sino organizarlas eficientemente en una composicion grande.

16 Lectura bibliografica aplicada al repo

16.1 Petersen–Zech

La lectura de Petersen–Zech entra aqui como marco formal: datos como variables aleatorias, hipotesis parametrizadas, riesgo poblacional, riesgo empirico, optimizacion no convexa y Transformers como clases de funciones compuestas. En el repo, esa teoria no se usa de forma abstracta sino asi:

$$\mathcal{D} \rightsquigarrow \text{proceso generador de texto}, \quad \hat{R} \rightsquigarrow \text{F.cross_entropy}, \quad \mathcal{H} \rightsquigarrow \{\text{bigram, transformer, llama}\}.$$

La advertencia teorica principal es que las ventanas solapadas no cumplen la version iid mas simple. Por eso el lenguaje de generalizacion debe usarse con cuidado: el test loss es una estimacion empirica bajo el split del repo, no una garantia universal.

16.2 Wegner

Wegner complementa el documento en tres lugares. Primero, el dataset se entiende como una matriz de observaciones, aunque aqui las filas sean ventanas de una secuencia dependiente. Segundo, maxima verosimilitud explica por que minimizar NLL es equivalente a ajustar una distribucion probabilistica. Tercero, alta dimension y compresion explican por que embeddings, SVD conceptual y representaciones latentes pertenecen al mismo lenguaje matematico.

En notacion de datos,

$$X = \begin{pmatrix} x_0 & x_1 & \cdots & x_{B-1} \\ x_1 & x_2 & \cdots & x_B \\ \vdots & \vdots & & \vdots \end{pmatrix}, \quad Y = \begin{pmatrix} x_1 & x_2 & \cdots & x_B \\ x_2 & x_3 & \cdots & x_{B+1} \\ \vdots & \vdots & & \vdots \end{pmatrix}.$$

Esta es la matriz de diseno del experimento, con la particularidad de que sus filas no son independientes.

16.3 Calin

Calin complementa las partes internas de deep learning: funciones de activacion, costos, entropia cruzada, KL, backpropagation, informacion y capacidad. En el repo esto aterriza en cuatro formulas:

$$\text{softmax}(z)_j = \frac{e^{z_j}}{\sum_k e^{z_k}}, \quad \ell(z, y) = -\log \text{softmax}(z)_y,$$

$$\text{SiLU}(a) = a\sigma(a), \quad \text{SwiGLU}(x) = W_d(\text{SiLU}(W_g x) \odot W_u x).$$

La lectura de capacidad no debe confundirse con tamano de parametros solamente: dos modelos con numeros parecidos de parametros pueden tener sesgos inductivos distintos por su forma de composicion, normalizacion y codificacion posicional.

16.4 Chang et al.

Chang et al. motivan la pregunta empirica: bajo restricciones de parametros, la calidad de datos puede cambiar la eficiencia del escalamiento. En este repo, la version operacional de esa idea es restringida:

calidad de datos $\rightsquigarrow$ diversidad empirica, entropia local, gzip, distinct-n.

El paper no autoriza concluir que toda diversidad sea buena. La lectura correcta para este experimento es condicional:

datos mas diversos son mas efectivos solo si esa diversidad reduce riesgo alcanzable bajo el presupuesto dado.

Cuando high es mas diverso pero tiene peor test loss, el resultado indica que las metricas de diversidad capturan dificultad adicional o variacion no aprovechada por el modelo, no que la teoria de calidad de datos sea falsa.

17 Trazas de dimensiones en una pasada forward

Esta seccion fija una notacion comun para leer el codigo sin ambigüedad. Sea

$$M = \text{batch size}, \quad T = \text{sequence length}, \quad B = \text{block size}, \quad m = |\mathcal{V}|, \quad d = \text{embedding dim.}$$

Durante entrenamiento usualmente $T = B$, aunque durante generacion puede ser $T \leq B$ por el truncamiento `idx[:, -block_size:]`.

17.1 Bigrama

Entrada:

$$X \in \{0, \dots, m-1\}^{M \times T}.$$

Parametro:

$$A \in \mathbb{R}^{m \times m}.$$

Salida:

$$Z = A[X] \in \mathbb{R}^{M \times T \times m}.$$

La notacion $A[X]$ significa indexacion fila por fila: $Z_{b,t,:} = A_{X_{b,t},:}$. Si hay targets,

$$Y \in \{0, \dots, m-1\}^{M \times T},$$

la perdida aplanada

$$Z \mapsto Z^b \in \mathbb{R}^{MT \times m}, \quad Y \mapsto Y^b \in \{0, \dots, m-1\}^{MT}.$$

Por tanto, cada posicion de cada secuencia del batch cuenta como una observacion de clasificacion multiclase.

17.2 Transformer pequeno

La entrada tokenizada produce dos tablas:

$$E[X] \in \mathbb{R}^{M \times T \times d}, \quad P[0:T] \in \mathbb{R}^{T \times d}.$$

Por broadcasting,

$$H^{(0)} = E[X] + P[0:T] \in \mathbb{R}^{M \times T \times d}.$$

En una cabeza de atencion,

$$Q, K, V \in \mathbb{R}^{M \times T \times d_h}, \quad QK^\top \in \mathbb{R}^{M \times T \times T},$$

donde la transposicion es solo en las dos ultimas dimensiones. La mascara causal tiene forma (T, T) y se aplica a cada elemento del batch. La salida de una cabeza es $\mathbb{R}^{M \times T \times d_h}$. Con H cabezas,

$$\text{concat}(O^{(1)}, \dots, O^{(H)}) \in \mathbb{R}^{M \times T \times Hd_h} = \mathbb{R}^{M \times T \times d}.$$

La proyeccion final preserva $\mathbb{R}^{M \times T \times d}$. El MLP/feedforward tambien preserva la dimension externa; solo cambia temporalmente a una dimension oculta. Por eso cada bloque Transformer es un endomorfismo de $\mathbb{R}^{M \times T \times d}$.

17.3 LLaMA-style

La entrada del modelo LLaMA-style empieza igual con embeddings de token:

$$H^{(0)} = E[X] \in \mathbb{R}^{M \times T \times d}.$$

No se suma una tabla P_t . La informacion posicional entra despues, al partir cabezas:

$$Q, K, V \in \mathbb{R}^{M \times H \times T \times d_h}.$$

RoPE exige d_h par porque rota coordenadas por pares. Despues de RoPE:

$$\tilde{Q}_{b,h,t,:} = R_t Q_{b,h,t,:}, \quad \tilde{K}_{b,h,s,:} = R_s K_{b,h,s,:}.$$

Los scores tienen forma

$$S \in \mathbb{R}^{M \times H \times T \times T}.$$

Tras softmax y mezcla con V , se obtiene

$$O \in \mathbb{R}^{M \times H \times T \times d_h}.$$

El codigo transpone y reordena:

$$O \mapsto O' \in \mathbb{R}^{M \times T \times H d_h} = \mathbb{R}^{M \times T \times d}.$$

La cabeza final vuelve a

$$Z \in \mathbb{R}^{M \times T \times m}.$$

Observacion 17.1 (Por que estas formas importan). *Muchos errores conceptuales en Transformers vienen de confundir tres ejes: batch, tiempo y canal. La atencion mezcla posiciones temporales, no ejemplos del batch. Las proyecciones lineales mezclan canales, no posiciones. La mascara actua sobre pares de posiciones, no sobre el vocabulario.*

18 Errores de interpretacion que el documento debe evitar

Proposicion 18.1 (Menor entropia de corpus no implica mejor corpus). *Si $\hat{H}_1(s_1) < \hat{H}_1(s_2)$, no se sigue que s_1 sea de mayor calidad. Solo se sigue que la distribucion marginal de caracteres de s_1 esta mas concentrada.*

Proof. La entropia marginal depende de frecuencias de caracteres. Un texto repetitivo como $aaaa \cdots a$ tiene entropia cercana a cero, pero no es un corpus rico. Por tanto, la calidad no puede identificarse con baja entropia marginal. $\square$

Proposicion 18.2 (Mayor distinct-n no implica menor perdida). *Si $\text{distinct}_n(s_2) > \text{distinct}_n(s_1)$, no se sigue que un modelo entrenado en s_2 alcance menor test loss bajo igual presupuesto.*

Proof. distinct_n mide proporcion de n-gramas unicos. Aumentar esa proporcion puede reducir repeticion util y aumentar la incertidumbre condicional del siguiente token. Si el modelo no puede aprender las regularidades de largo alcance que explican esa variedad, la perdida predictiva puede aumentar. $\square$

Proposicion 18.3 (Perplejidad no es calidad semantica). *La perplejidad*

$$\text{PPL} = \exp(\hat{L})$$

es una transformacion monotona de la perdida promedio. No mide directamente verdad, coherencia global ni valor pedagogico del texto generado.

Observacion 18.4. *En este repo, la perplejidad sirve para comparar modelos bajo el mismo tokenizer, split y objetivo. Compararla entre tokenizadores distintos o corpus con vocabularios incompatibles seria matematicamente delicado.*

Proposicion 18.5 (Un Transformer pequeno no prueba el limite de la arquitectura). *Si el `TinyTransformerLanguageModel` no mejora sobre el bigrama en un experimento concreto, no se concluye que los Transformers no sirvan para el corpus. Se concluye solamente algo sobre esta clase parametrica, esta escala y este presupuesto de optimizacion.*

Proof. La familia de Transformers contiene modelos con distintos d , numero de capas, numero de cabezas, contexto y entrenamiento. El experimento fija un punto pequeno en esa familia. Un resultado negativo local no identifica el comportamiento de toda la clase. $\square$

Observacion 18.6 (Conexion con statistical learning theory). *Estas proposiciones son aplicaciones del mismo principio: una metrica observada es una funcion parcial del fenomeno. El riesgo de test, la entropia, distinct-n y gzip ratio son proyecciones distintas del objeto real. La inferencia rigurosa consiste en decir exactamente que proyeccion se midio y que conclusiones no estan autorizadas por esa medicion.*

19 Limitaciones matematicas

1. El modelo es character-level; no hay tokens subpalabra ni semantica explicita.
2. Las ventanas se solapan; no hay muestra iid pura.
3. El test set es un bloque contiguo; puede haber desplazamiento temporal de distribucion.
4. Las metricas de diversidad son locales o compresivas; no miden syntheticity ni calidad semantica.
5. La expresividad teorica de Transformers no implica aprendibilidad con 2 capas, $d = 64$, batch size 32 y presupuesto limitado.
6. La comparacion medium/high es empirica y pequena; no prueba una ley general sobre diversidad.

20 Resumen de correspondencias

Modulo	Parametros aprendidos	Transformacion
BigramLanguageModel	$A \in \mathbb{R}^{m \times m}$	$i \mapsto \text{softmax}(A_{i,:})$
CausalSelfAttentionHead	W_Q, W_K, W_V	$\text{softmax}(QK^\top / \sqrt{d_h} + M)V$
TinyTransformerLanguageModel	embeddings, MHA, FFN, LN, lm head	$X \mapsto Z_\theta \in \mathbb{R}^{M \times T \times m}$
LlamaStyleLanguageModel	embeddings, RMSNorm, RoPE attention, SwiGLU	Transformer causal con posicion rotatoria
corpus_stats.py	ninguno	estimadores empiricos de diversidad
train.py	todos los parametros del modelo	minimizacion de entropia cruzada con AdamW

References

- [1] Ovidiu Calin. *Deep Learning Architectures: A Mathematical Approach*. Springer Series in the Data Sciences. Springer, Cham, 2020.
- [2] Ernie Chang, Matteo Paltenghi, Yang Li, Pin-Jie Lin, Changsheng Zhao, Patrick Huber, Zechun Liu, Rastislav Rabatin, Yangyang Shi, and Vikas Chandra. Scaling parameter-constrained language models with quality data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 80–97, Miami, Florida, US, 2024. Association for Computational Linguistics.
- [3] Philipp Petersen and Jakob Zech. *Mathematical theory of deep learning*, 2026.
- [4] Sven A. Wegner. *Mathematical Introduction to Data Science*. Springer, Berlin, Heidelberg, 2024.